PySpark Column Functions

Coding Assignment — Module 3

20 Questions | String • Date & Time • Aggregation • Conditional • Complex Types

Course	Module	Trainer	Total Marks
PySpark for Data Engineering	Column Functions — Module 3	Sateesh (MCT, 22+ yrs)	45 pts + 5 Bonus = 50 pts

Assignment Instructions

- Use the provided dataset for all questions unless a question specifies its own data.
- Import all required functions from pyspark.sql.functions at the top of your notebook.
- Use withColumn() to add computed columns unless the question asks for select().
- Use .show() to display outputs and .printSchema() where schema verification is asked.
- Submit your Colab/Jupyter notebook with all cells executed and outputs visible.
- Easy = 1 pt | Medium = 2 pts | Hard = 3 pts | Bonus = 5 pts | Total = 50 pts

Sateesh Says
Module 3 is pure gold for interviews! Every company — TCS, Infosys, Wipro, Cognizant — asks string and date function questions in their PySpark rounds. lower(), date_add(), when() — these 3 alone appear in 80% of Databricks interview problems.

Dataset — Use This for All Questions

Dataset 1: LevelUp Student Records (students_df)

This is the primary dataset for all String and Date function questions.

students_data — Preview

student_id	first_name	last_name	city	course	dob	enrollment_date	fees_paid	is_active

1	Arjun	Sharma	Hyderabad	Databricks	1998-05-14	2024-01-10	12000	true
2	Priya	Reddy	Bangalore	PySpark	1995-11-22	2024-02-05	8000	true
3	Vikram	Nair	Mumbai	GCP	1992-03-08	2024-02-20	10000	false
4	Neha	Gupta	Delhi	Fabric	2000-07-30	2024-03-15	9000	true
5	Rahul	Iyer	Hyderabad	Databricks	1990-01-19	2024-03-28	12000	true
6	Ananya	Patel	Pune	Snowflake	1997-09-05	2024-04-10	7500	false
7	Karthik	Rao	Bangalore	GCP	1993-12-25	2024-04-22	10000	true
8	Deepika	Singh	Chennai	Python	1999-06-17	2024-05-03	6000	true
9	Suresh	Kumar	Hyderabad	PySpark	1988-08-11	2024-05-20	8000	false
10	Meera	Joshi	Mumbai	Databricks	1996-04-02	2024-06-01	12000	true

Note: first_name column has intentional leading/trailing spaces for trim exercises.

```
from pyspark.sql import SparkSession
from pyspark.sql.functions import col, lower, upper, length, substring,
split,
    trim, ltrim, rtrim, lpad, rpad, concat, concat_ws, lit,
    current_date, current_timestamp, date_add, date_sub, datediff,
    months_between, add_months, next_day, last_day, date_format,
    dayofyear, dayofmonth, dayofweek, year, month,
    count, countDistinct, sum, avg, min, max, when, array, map_from_arrays,
struct

spark = SparkSession.builder.appName('LevelUpModule3').getOrCreate()

students_data = [
  (1, ' Arjun',  'Sharma',  'Hyderabad', 'Databricks', '1998-05-14', '2024-
01-10', 12000, True ),
```

```
  (2, 'Priya ',  'Reddy',   'Bangalore', 'PySpark',    '1995-11-22', '2024-
02-05',  8000, True ),
  (3, 'Vikram',  'Nair',    'Mumbai',    'GCP',        '1992-03-08', '2024-
02-20', 10000, False),
  (4, 'Neha',    'Gupta',   'Delhi',     'Fabric',     '2000-07-30', '2024-
03-15',  9000, True ),
  (5, 'Rahul',   'Iyer',    'Hyderabad', 'Databricks', '1990-01-19', '2024-
03-28', 12000, True ),
  (6, ' Ananya', 'Patel',   'Pune',      'Snowflake',  '1997-09-05', '2024-
04-10',  7500, False),
  (7, 'Karthik', 'Rao',     'Bangalore', 'GCP',        '1993-12-25', '2024-
04-22', 10000, True ),
  (8, 'Deepika', 'Singh',   'Chennai',   'Python',     '1999-06-17', '2024-
05-03',  6000, True ),
  (9, 'Suresh ', 'Kumar',   'Hyderabad', 'PySpark',    '1988-08-11', '2024-
05-20',  8000, False),
 (10, 'Meera',   'Joshi',   'Mumbai',    'Databricks', '1996-04-02', '2024-
06-01', 12000, True ),
]

stu_cols = ['student_id','first_name','last_name','city','course',
            'dob','enrollment_date','fees_paid','is_active']

students_df = spark.createDataFrame(students_data, stu_cols)
```

Dataset 2: Sales Data (for Aggregation Questions)

sales_data — Preview

sale_id	salesperson	region	product	quantity	unit_price	sale_date
S01	Arjun Sharma	South	Databricks Course	2	12000	2024-01-15
S02	Priya Reddy	North	PySpark Course	3	8000	2024-01-20
S03	Vikram Nair	West	GCP Course	1	10000	2024-02-10
S04	Neha Gupta	North	Fabric Course	2	9000	2024-02-25
S05	Arjun Sharma	South	PySpark Course	4	8000	2024-03-05
S06	Priya Reddy	North	Databricks Course	1	12000	2024-03-18
S07	Karthik Rao	South	GCP Course	2	10000	2024-04-02
S08	Deepika Singh	East	Snowflake Course	3	7500	2024-04-20

```
sales_data = [
  ('S01','Arjun Sharma',  'South','Databricks Course', 2,12000,'2024-01-15'),
  ('S02','Priya Reddy',   'North','PySpark Course',    3, 8000,'2024-01-20'),
  ('S03','Vikram Nair',   'West', 'GCP Course',        1,10000,'2024-02-10'),
  ('S04','Neha Gupta',    'North','Fabric Course',     2, 9000,'2024-02-25'),
  ('S05','Arjun Sharma',  'South','PySpark Course',    4, 8000,'2024-03-05'),
  ('S06','Priya Reddy',   'North','Databricks Course', 1,12000,'2024-03-18'),
  ('S07','Karthik Rao',   'South','GCP Course',        2,10000,'2024-04-02'),
  ('S08','Deepika Singh', 'East', 'Snowflake Course',  3, 7500,'2024-04-20'),
```

```
]
sale_cols =
['sale_id','salesperson','region','product','quantity','unit_price','sale_dat
e']
sales_df = spark.createDataFrame(sales_data, sale_cols)
```

□ Assignment Questions

Section 1: String Manipulation Functions [Easy–Medium]

Q1. **[Easy]** | *lower(), upper()*

Using students_df:
(a) Add column 'first_name_lower' using lower(col('first_name'))
(b) Add column 'course_upper' using upper(col('course'))
(c) Select: student_id, first_name, first_name_lower, course, course_upper
Show the result.

Q2. **[Easy]** | *length() and substring()*

Using students_df:
(a) Add column 'name_length' = length of first_name (including spaces)
(b) Add column 'name_initial' = first 1 character of first_name using substring(col, 1, 1)
(c) Add column 'course_code' = first 3 characters of course using substring()
Show: student_id, first_name, name_length, name_initial, course_code

Q3. **[Easy]** | *trim(), ltrim(), rtrim()*

The first_name column in students_df has intentional leading/trailing spaces.
(a) Add 'name_trimmed' using trim() — removes both sides
(b) Add 'name_ltrimmed' using ltrim() — removes left spaces only
(c) Add 'name_rtrimmed' using rtrim() — removes right spaces only
Show: student_id, first_name, name_trimmed, name_ltrimmed, name_rtrimmed
Verify the difference in output for rows with leading vs trailing spaces.

Q4. **[Easy]** | *lpad() and rpad()*

Using students_df:
(a) Add 'padded_id' = lpad(student_id cast to string, 5, '0') → e.g. '00001'
(b) Add 'course_padded' = rpad(course, 15, '-') → e.g. 'Databricks-----'
Show: student_id, padded_id, course, course_padded
Hint: cast student_id to string first using cast('string')

Q5. **[Easy]** | *concat() and concat_ws()*

Using students_df:
(a) Create 'full_name' using concat(first_name, lit(' '), last_name)
(b) Create 'full_name_ws' using concat_ws(' ', trim(col('first_name')), col('last_name'))
(c) Create 'student_label' using concat_ws(' | ', col('student_id').cast('string'), col('course'), col('city'))
Example output: '1 | Databricks | Hyderabad'
Show: student_id, full_name, full_name_ws, student_label

Q6. **[Medium]** | *split()*

Using students_df:
(a) Add column 'dob_parts' = split(col('dob'), '-') → produces an array [year, month, day]
(b) Add column 'dob_year_str' = dob_parts[0] (first element of split array)
(c) Add column 'dob_month_str' = dob_parts[1] (second element)
(d) Show: student_id, dob, dob_parts, dob_year_str, dob_month_str
Hint: After split(), access elements using col('dob_parts')[0], col('dob_parts')[1]

Sateesh Says
String functions are bread and butter of every ETL pipeline. At Infosys, first step of any customer data pipeline is: trim → lower → concat full_name → lpad IDs for zero-filling. You just coded all four steps in Q1–Q5!

Section 2: Date & Time Functions [Medium–Hard]

Q7. **[Medium]** | *current_date() and current_timestamp()*

Using students_df:
(a) Add column 'today' using current_date()
(b) Add column 'now' using current_timestamp()
(c) Add column 'days_since_enrollment' = datediff(current_date(), col('enrollment_date'))
Show: student_id, first_name, enrollment_date, today, days_since_enrollment
Note: datediff(end_date, start_date) returns number of days between the two dates.

Q8. **[Medium]** | *date_add() and date_sub()*

Using students_df:
(a) Add 'cert_due_date' = enrollment_date + 90 days using date_add()
(b) Add 'early_bird_date' = enrollment_date - 7 days using date_sub()
(c) Show: student_id, first_name, enrollment_date, cert_due_date, early_bird_date

Q9. **[Medium]** | *months_between() and add_months()*

Using students_df:
(a) Add 'age_in_months' = months_between(current_date(), col('dob')) — cast to int
(b) Add 'dob_plus_3months' = add_months(col('dob'), 3)
(c) Add 'age_years' = (age_in_months / 12).cast('int')
Show: student_id, first_name, dob, age_in_months, age_years, dob_plus_3months

Q10. **[Medium]** | *next_day() and last_day()*

Using students_df:
(a) Add 'next_sunday' = next day after enrollment_date that is a Sunday, using next_day(col('enrollment_date'), 'Sunday')
(b) Add 'month_end' = last day of the enrollment month, using last_day(col('enrollment_date'))
Show: student_id, enrollment_date, next_sunday, month_end

Q11. **[Medium]** | *dayofyear(), dayofmonth(), dayofweek(), year(), month()*

Using students_df, extract all date parts from the dob column:
(a) 'birth_year' = year(col('dob'))
(b) 'birth_month' = month(col('dob'))
(c) 'birth_day' = dayofmonth(col('dob'))
(d) 'birth_dayofwk' = dayofweek(col('dob')) → 1=Sunday, 7=Saturday
(e) 'birth_dayofyr' = dayofyear(col('dob'))
Show all 5 new columns alongside student_id, first_name, dob.

Q12. **[Hard]** | *date_format()*

Using students_df, use date_format() to format enrollment_date in multiple ways:
(a) 'fmt_yyyyMMdd' = date_format(col('enrollment_date'), 'yyyyMMdd') → '20240110'
(b) 'fmt_readable' = date_format(col('enrollment_date'), 'dd-MMM-yyyy') → '10-Jan-2024'
(c) 'fmt_int' = date_format(col('enrollment_date'), 'yyyyMMdd').cast('int')
(d) 'quarter' = date_format(col('enrollment_date'), 'QQQ') → 'Q1'
Show: student_id, enrollment_date, fmt_yyyyMMdd, fmt_readable, fmt_int, quarter

Sateesh Says
date_format is Swiggy's secret weapon — every order is logged with a timestamp and then formatted as 'yyyyMMdd' for daily partitioning on S3/ADLS. Q12 teaches you exactly how Databricks pipelines partition their Delta Lake tables by date!

Section 3: Aggregation Functions [Medium–Hard]

Q13. **[Medium]** | *count(), countDistinct(), groupBy().agg()*

Using students_df:
(a) Count total rows: students_df.count()
(b) Count distinct cities: students_df.select('city').distinct().count()
(c) Group by 'course' and count number of students per course:
students_df.groupBy('course').agg(count('first_name').alias('student_count')).show()
(d) Group by 'city' and count distinct courses offered in each city:
use countDistinct('course').alias('course_variety')
Show all four results.

Q14. **[Medium]** | *sum(), avg(), min(), max() with groupBy*

Using students_df:
(a) Global aggregations (no groupBy):
- Total fees collected: df.agg(sum('fees_paid').alias('total_fees'))
- Average fees: avg('fees_paid')
- Min fees: min('fees_paid')
- Max fees: max('fees_paid')

(b) Group by 'course' and compute sum, avg, min, max of fees_paid in one .agg() call
Show both results.

Q15. **[Hard]** | *Multi-Column groupBy + Multiple Aggregations*

Using sales_df:
Group by BOTH 'region' AND 'salesperson' and compute in one .agg() call:
(a) total_sales_qty = sum('quantity')
(b) total_revenue = sum(col('quantity') * col('unit_price')) — computed inline
(c) avg_unit_price = avg('unit_price')
(d) num_transactions = count('sale_id')
Sort the result by total_revenue descending. Show the final grouped DataFrame.
Hint: for computed aggregations, use F.sum(col('quantity') * col('unit_price'))

Sateesh Says
Every Databricks Silver→Gold pipeline is basically Q13-Q15. Zomato computes: group by city + restaurant, then sum(orders), avg(delivery_time), max(order_value). That is Q15 — verbatim. You are now writing production Gold-layer code!

Section 4: Conditional Functions — when().otherwise() [Medium–Hard]

Q16. **[Medium]** | *Basic when().otherwise()*

Using students_df, add a column 'age_group':

- If age < 28 → 'Junior'
- If age >= 28 and < 33 → 'Mid'
- If age >= 33 → 'Senior'

Hint: compute age first using: (months_between(current_date(), col('dob'))/12).cast('int')
Use nested when().when().otherwise() for three conditions.
Show: student_id, first_name, dob, age, age_group

Q17. **[Medium]** | *when() for Flag Columns*

Using students_df:
(a) Add 'status_label' using when():
- is_active = True → 'Active'
- is_active = False → 'Inactive'

(b) Add 'fees_tier' using when():
- fees_paid >= 10000 → 'Premium'
- fees_paid >= 8000 → 'Standard'
- Otherwise → 'Basic'

Show: student_id, first_name, course, fees_paid, fees_tier, is_active, status_label

Q18. **[Hard]** | *when() + withColumn + concat Combined*

Using students_df, build a 'student_profile' column that combines multiple conditions:
Format: '<TrimmedName> | <City> | <Course> | <Tier>'
Where Tier = 'PREMIUM' if fees_paid >= 10000 else 'STANDARD'
Steps:
1. Add 'clean_name' = concat_ws(' ', trim(col('first_name')), col('last_name'))
2. Add 'tier' = when(col('fees_paid') >= 10000, 'PREMIUM').otherwise('STANDARD')
3. Add 'student_profile' = concat_ws(' | ', col('clean_name'), col('city'), col('course'), col('tier'))

Show: student_id, student_profile

Section 5: Complex Type Functions & Full Pipeline [Hard]

Q19. **[Hard]** | *array(), map_from_arrays(), struct()*

Using students_df:
(a) Add 'name_array' = array(trim(col('first_name')), col('last_name'))
→ produces: ['Arjun', 'Sharma']

(b) Add 'name_map' = map_from_arrays(
array(lit('first'), lit('last')),
array(trim(col('first_name')), col('last_name'))
) → produces: {'first': 'Arjun', 'last': 'Sharma'}

(c) Add 'name_struct' = struct(trim(col('first_name')).alias('fn'), col('last_name').alias('ln'))

→ produces: {fn: 'Arjun', ln: 'Sharma'}
Show: student_id, name_array, name_map, name_struct
Print the schema to confirm ArrayType, MapType, StructType.

Q20. **[Hard]** | *Full Column Functions Pipeline*

Using students_df, build a complete analytical pipeline in one chained statement:
Step 1: trim first_name → 'clean_name'
Step 2: concat clean_name + ' ' + last_name → 'full_name'
Step 3: upper(course) → 'course_upper'
Step 4: datediff(current_date(), enrollment_date) → 'days_enrolled'
Step 5: (months_between(current_date(), dob)/12).cast('int') → 'age'
Step 6: when(age < 30, 'Young').when(age < 35, 'Mid').otherwise('Senior') → 'age_group'
Step 7: when(fees_paid >= 10000, 'Premium').otherwise('Standard') → 'tier'
Step 8: concat_ws(' | ', full_name, col('city'), course_upper, tier) → 'profile'
Final select: student_id, full_name, age, age_group, days_enrolled, tier, profile
Show the final result.

BONUS QUESTION [+5 Points]

Mini Project: LevelUp Student Analytics Dashboard

Using students_df, build a complete analytics report with these steps:

1. Clean Data: trim first_name, create full_name using concat_ws, cast student_id to string with lpad(5,'0') for zero-padded ID.
2. Date Analysis: extract birth_year, birth_month from dob. Compute age_years using months_between. Compute days_since_joining using datediff from enrollment_date.
3. Conditional Columns: Add age_group (Junior/Mid/Senior), fees_tier (Basic/Standard/Premium), and active_status ('Active'/'Inactive') using when().otherwise().
4. Aggregation Report: Group by course and compute — count of students, total fees collected, average age, min and max fees. Show this summary.
5. Date Formatting: Create formatted_enrollment = date_format(enrollment_date, 'dd-MMM-yyyy') and enrollment_quarter = date_format(enrollment_date, 'QQQ').
6. Complex Types: Add a 'name_struct' column using struct(clean_first_name, last_name) and a 'skills_array' column using array(lit('PySpark'), lit('SQL'), col('course')).
7. Final Profile: Create a 'student_card' column as: 'ID: <padded_id> | <full_name> | <city> | <course> | <tier> | <active_status>'
8. Register as temp view 'student_report' and run a spark.sql() to find top city by total fees collected.

Scoring Rubric

Difficulty	Questions	Points Each	Question Nos.	Total
Easy	6	1 pt	Q1, Q2, Q3, Q4, Q5, Q6	6 pts
Medium	8	2 pts	Q7, Q8, Q9, Q10, Q11, Q12, Q13, Q14	16 pts
Hard	6	3 pts	Q15, Q16, Q17, Q18, Q19, Q20	18 pts
Bonus	1	5 pts	Mini Project	5 pts
TOTAL	**21**	**—**	**—**	**45 pts**

Sateesh Says
Q20 and the Bonus together is a real Databricks notebook — the kind TCS iON and Infosys BPM teams use daily. If you complete the entire assignment independently, add it to your GitHub and LinkedIn. It is your first PySpark project portfolio piece!

All the Best! — Sateesh | LevelUp Cloud Trainings
www.levelupedu.net | Hyderabad, India